

# Verified Online Constraint-Program Repair for Recoverable Robotic Manipulation

Method, Implementation, CPU Evidence, and GPU Integration Plan

Internal Technical Report — Coauthor Handoff

July 29, 2026

## Abstract

This report documents the complete state of the REKEP-R project: an online repair layer for ReKep relational-keypoint constraint programs that detects an interruption, captures a continuation of the actively executing stage, *synthesizes* a repair program at runtime from atomic typed operators, *verifies* it by sequential rollout, splices it into the live task graph, and then restores and resumes the interrupted stage. We report a frozen CPU implementation, six theoretical results with explicit assumptions, and  $\sim 59,000$  simulated episodes of evidence. The headline finding is deliberately bounded: when the exact recovery branch is precompiled, a strong parameterized offline baseline is *exactly equal* to online repair (0/0 discordant pairs,  $p = 1.0$ ). The online advantage lies in compact operator storage, runtime structural adaptability, and coverage under partial precompilation. No GPU code has been executed.

## Contents

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>1</b> | <b>Executive summary</b>                                  | <b>3</b>  |
| <b>2</b> | <b>Research origin and contribution context</b>           | <b>3</b>  |
| <b>3</b> | <b>Original research outline</b>                          | <b>3</b>  |
| <b>4</b> | <b>Formal problem formulation</b>                         | <b>4</b>  |
| <b>5</b> | <b>Method</b>                                             | <b>5</b>  |
| 5.1      | Layers . . . . .                                          | 5         |
| 5.2      | Three paradigms . . . . .                                 | 6         |
| <b>6</b> | <b>Correctness and theoretical results</b>                | <b>6</b>  |
| 6.1      | Simultaneous-objective conflict . . . . .                 | 6         |
| 6.2      | Synthesis-layer results . . . . .                         | 7         |
| 6.3      | Three levels of validity . . . . .                        | 8         |
| <b>7</b> | <b>Code architecture</b>                                  | <b>8</b>  |
| 7.1      | End-to-end call graph . . . . .                           | 9         |
| <b>8</b> | <b>Bugs and research-validity issues already resolved</b> | <b>9</b>  |
| <b>9</b> | <b>CPU experimental protocol</b>                          | <b>10</b> |

|                                           |           |
|-------------------------------------------|-----------|
| <b>10 CPU results</b>                     | <b>11</b> |
| 10.1 Strong-baseline comparison . . . . . | 11        |
| 10.2 Coverage-storage trade-off . . . . . | 11        |
| 10.3 Ablations . . . . .                  | 11        |
| 10.4 Model mismatch . . . . .             | 13        |
| <b>11 Frozen paper claims</b>             | <b>14</b> |
| <b>12 Current limitations</b>             | <b>14</b> |
| <b>13 GPU phase plan</b>                  | <b>15</b> |
| <b>14 ICRA experiment matrix</b>          | <b>16</b> |
| <b>15 Handoff checklist</b>               | <b>16</b> |

# 1 Executive summary

## Research question.

How can a robot modify the structure of an actively executing ReKep constraint program online, in response to an interruption, and then safely restore and resume the original task?

## Central distinction.

---

|                                     |                                                                         |
|-------------------------------------|-------------------------------------------------------------------------|
| <b>offline precompiled recovery</b> | select an existing recovery structure                                   |
| <b>online repair</b>                | synthesize, verify, and splice a <i>new</i> repair structure at runtime |

---

**What the evidence supports.** Exact precompiled coverage *matches* online performance whenever the exact branch is available. Across 16,200 paired episodes the strong parameterized offline baseline and online repair are identical on all three success measures (0.973 safe delivery, 0.973 collision-free recovery, 0.683 task success), with zero discordant pairs and  $p = 1.0$  under an exact McNemar test.

The online advantage is therefore **not universal superiority**. It is:

1. **compact operator representation** — 9 atomic operators and zero compilation, versus 17 precompiled branches / 109 graph nodes / 157 offline search expansions for exact coverage;
2. **runtime structural adaptability** — a repair sequence never stored before execution can be constructed;
3. **coverage under partial precompilation** — +0.380 [+0.362, +0.399] safe delivery over an atomic-coverage offline baseline, concentrated entirely in compound events.

**Status.** The CPU architecture, benchmark, claims, and tests are **frozen**. GPU/ReKep integration is prepared but **not executed**.

# 2 Research origin and contribution context

**Internal note.** The abstract idea of *online repair* — that a disturbance may require changing the *structure* of the executing task program rather than only re-optimizing a trajectory inside a fixed stage — **originated from the collaborator**. From that starting point the project has been **jointly developed** into a formal method, an implementation, a set of theoretical results, and an experimental program.

**Final authorship, author order, and CRediT contribution statements are not decided and will be confirmed jointly by the coauthors before submission.** No ordering, percentages, or role assignments are recorded anywhere in this handoff, by design.

Open questions requiring joint agreement are listed in `COLLABORATION_NOTES.md`.

# 3 Original research outline

The intended runtime pipeline, which the implementation now realizes:

```

nominal ReKep program
-> runtime event detection
-> continuation capture
-> abstract repair-goal construction
-> online operator synthesis
-> structural legality checking
-> sequential rollout verification
-> candidate selection
-> active-program splice
-> restoration validation
-> resumed nominal execution

```

**Why this is program repair, not trajectory replanning.** Trajectory replanning searches for a new continuous path *within a fixed discrete stage structure*: the set of active objectives, their ordering, and the stage boundaries are unchanged. Program repair changes that discrete structure itself — which objectives are active, in what order, and where the boundaries lie — and then hands the result to the *unchanged* trajectory optimizer.

Three consequences distinguish the two:

1. *Objective deactivation.* A path-only reaction must satisfy the nominal and interruption objectives simultaneously and pays the irreducible conflict floor of Theorem 1. Repair deactivates the nominal objective for a bounded interval.
2. *Ordered commitments.* Recoveries such as `Retreat`  $\prec$  `WaitUntilClear`  $\prec$  `ReacquireTarget` are temporal orderings over modes, not geometric constraints, and have no representation inside one stage.
3. *Restoration.* After the recovery the system must return to a state satisfying the *interrupted stage's* entry condition — a program-level contract, not a trajectory property.

## 4 Formal problem formulation

**Nominal ordered constraint program.** A task is an ordered sequence of stages  $\mathcal{S} = (s_1, \dots, s_N)$ , each

$$s_i = (\mathcal{C}_{\text{sub}}^{(i)}, \mathcal{C}_{\text{path}}^{(i)}, \rho_i, m_i, \psi_i), \quad (1)$$

with subgoal constraints  $\mathcal{C}_{\text{sub}}$  (hold at the stage boundary), path constraints  $\mathcal{C}_{\text{path}}$  (hold throughout), transition guard  $\rho_i$ , semantic mode  $m_i$ , and metadata  $\psi_i$ . A relational keypoint constraint is  $f : \mathbb{R}^{K \times 3} \rightarrow \mathbb{R}$ , satisfied when  $f(k_t) \leq 0$ .

**Runtime state and stage lifecycle.** The runtime state is the pair (execution context, stage state), with

PENDING, ACTIVE, SUSPENDED, REPAIR\_ACTIVE, RESUMED, COMPLETED, FAILED.

**Event predicates.** The world exposes a predicate vector  $P_t \in \{0, 1\}^{K_p}$ ; currently `{obstacle_present, object_slipping}` plus a clearance flag. *Every* method receives exactly this vector, so no method observes a richer world than another.

**Continuation.** On interruption of the active stage  $s_i$  we capture

$$\kappa_t = (i, x_t, e_t, A_t, g_i, \mathcal{C}_i, e_{\text{handoff}}, \text{Pre}(s_i^{\text{resumed}}), \chi_t), \quad (2)$$

i.e. stage identity, state, end-effector pose, attachment state, the nominal target, the active constraints, a handoff pose, the resume contract, and a progress marker.

**Abstract state and repair operators.** Repair planning happens in a finite abstract space  $S = \{0, 1\}^d$ ,  $d = 11$ , over facts such as `nominal_suspended`, `object_grasped`, `object_stable`, `safe_clearance`, `ee_at_handoff`, `aligned_to_current_target`, `restore_valid`. An operator is a triple

$$o = (\text{pre}_o, \text{eff}_o, c_o), \quad \text{pre}_o : S \rightarrow \{0, 1\}, \quad \text{eff}_o : S \rightarrow S, \quad c_o > 0. \quad (3)$$

The library  $O$  contains `Suspend`, `Stabilize`, `Retreat`, `WaitUntilClear`, `ReacquireTarget`, `UpdateTargetContract`, `Realign`, `Resume`, `SafeStop`.

**RepairGoal.** Event requirements are first class. From the active predicate set we build

$$G = (T, F), \quad \text{sat}_G(s) \iff \left( \bigwedge_{p \in T} s.p \right) \wedge \left( \bigwedge_{p \in F} \neg s.p \right), \quad (4)$$

as the *union* of every active predicate's requirements plus `restore_valid`. For example

$$\text{obstacle: } \neg \text{obstacle\_present} \wedge \text{safe\_clearance}, \quad (5)$$

$$\text{slip: } \text{object\_grasped} \wedge \text{object\_stable}, \quad (6)$$

$$\text{relocation: } \text{aligned\_to\_current\_target}. \quad (7)$$

**Candidate repair program and selection.** A candidate is an operator sequence  $\pi = (o_1, \dots, o_m)$  instantiated into concrete stages. Let  $\Omega_t$  be the candidates that survive *structural legality* (acyclicity, presence of a restore edge, attachment consistency) and *concrete rollout feasibility* (no predicted collision, no guard timeout, within the episode horizon, valid attachment, zero requirement residual, restore contract reachable). Selection minimizes a state-dependent cost computed *from the rollout*:

$$\pi^* = \arg \min_{\pi \in \Omega_t} \left[ w_r |\text{res}(\pi)| + w_s [\bar{d}(\sigma) - \hat{d}_{\min}(\pi)]_+ + w_t \hat{T}(\pi) + w_h \hat{E}_{\text{ho}}(\pi) + w_a \mathbb{K}[\text{released}] + \alpha |\pi| + w_{\text{sw}} (|\pi| - 1) \right], \quad (8)$$

where  $\text{res}$  is the requirement residual,  $\hat{d}_{\min}$  the predicted minimum clearance,  $\bar{d}(\sigma)$  a severity-scaled desired standoff,  $\hat{T}$  the predicted duration and  $\hat{E}_{\text{ho}}$  the predicted handoff error. Candidates with  $\text{res} \neq \emptyset$  are *rejected*, not merely penalized; the residual term in (8) only ranks survivors.

**Repaired program.** The splice replaces the edge  $s_i \rightarrow s_{i+1}$  by

$$s_i^{\text{suspended}} \rightarrow u_1 \rightarrow \dots \rightarrow u_m \rightarrow s_i^{\text{resumed}}(\kappa_t) \rightarrow s_{i+1}. \quad (9)$$

## 5 Method

### 5.1 Layers

1. **TaskProgram IR.** Ordered stages plus an explicit `TaskGraph`; the solver only ever asks for `active_stage()`.
2. **Stage state machine.** Enforces (not by convention) that the interrupted stage becomes `SUSPENDED`, never `COMPLETED`; that  $s_{i+1}$  is unreachable until restore validates; and that `advance()` is *atomic* — all legality checks precede any mutation, so a blocked transition leaves state bit-identical.
3. **Continuation capture.**  $\kappa_t$  as above.
4. **First-class event requirements.** `RepairGoal` built from the full predicate vector; compound events take the union.

5. **Symbolic operators.** Preconditions encode physical semantics: `Stabilize` requires `object_grasped` (a released object cannot be reoriented by the gripper); `WaitUntilClear` requires a safe standoff and does *not* open clearance — only `Retreat` does. Operator semantics are deliberately non-overlapping.
6. **Bounded best-first search.** Uniform-cost over  $O$ ; each operator used at most once and required to change the state, so search terminates (Prop. 2). Observed mean: 15.2 expansions.
7. **Globally unique repair instances.** Node IDs are namespaced `stage::event::instance::operator#id` so repeated repairs in one episode cannot collide or create cycles.
8. **Structural legality filter.** Acyclicity, `HasRestore`, attachment consistency.
9. **Sequential rollout verifier.** Propagates state operator-by-operator under the planning model, using the *same* action-clipping utility as the environment, and independently re-derives the requirement residual from the predicted trajectory.
10. **Candidate scoring.** Equation (8).
11. **Program splice.** Equation (9).
12. **Restoration and resumption.** Resume’s exit guard *is* the resume contract; entry to  $s_i^{\text{resumed}}$  is gated on it.
13. **Safe fallback.** If no candidate survives, a bounded safe stop.
14. **Provenance.** Every action carries `action ← stage ← repair ← event ← continuation`.

## 5.2 Three paradigms

|                   | precompiled<br>branch                        | full | runtime template in-<br>stantiation  | runtime<br>synthesis | operator                 |
|-------------------|----------------------------------------------|------|--------------------------------------|----------------------|--------------------------|
| stored            | complete branch per<br>predicate combination | per  | complete sequence per<br>event class | <b>atomic</b>        | <b>operators</b>         |
| runtime decision  | which branch to acti-<br>vate                |      | which template to in-<br>stantiate   | <b>only</b>          | <b>which sequence to</b> |
| graph rewritten   | no                                           |      | yes                                  | yes                  | <b>construct</b>         |
| novel composition | no                                           |      | no                                   | <b>yes</b>           |                          |
| storage growth    | with covered combina-<br>tions               |      | with event classes                   | <b>constant</b>      |                          |

Critically, the strong offline baseline used in all experiments has *every* runtime capability online repair has — same predicates, parameter binding, controllers, rollout verifier, scoring, continuation capture, splice, restore — and differs *only* in that it cannot construct a sequence that was not compiled before execution.

## 6 Correctness and theoretical results

### 6.1 Simultaneous-objective conflict

**Theorem 1** (Conflict lower bound). *Let  $A, B \subseteq \mathcal{X}$  be nonempty with  $\delta = \text{dist}(A, B) > 0$ , and fix  $\lambda, \mu > 0$ . Then for  $J(x) = \lambda d(x, A)^2 + \mu d(x, B)^2$ ,*

$$\inf_{x \in \mathcal{X}} J(x) \geq \frac{\lambda\mu}{\lambda + \mu} \delta^2. \quad (10)$$

*Proof sketch.*  $d(x, A) + d(x, B) \geq \delta$  for any nonempty sets. Minimizing  $\lambda a^2 + \mu b^2$  subject to  $a + b \geq \delta$ ,  $a, b \geq 0$  is convex with active constraint; the optimum is  $a^* = \mu\delta/(\lambda + \mu)$ ,  $b^* = \lambda\delta/(\lambda + \mu)$ , giving the stated value.  $\square$

We write  $\inf$ , not  $\min$ :  $A, B$  need not be closed.

**Corollary 1** (Threshold failure, *fixed weights*). *Fix  $\lambda, \mu > 0$ . If success requires  $J(x) \leq \varepsilon$  with  $\varepsilon < \frac{\lambda\mu}{\lambda+\mu}\delta^2$ , no fixed-stage simultaneous optimum with those weights succeeds.*

**Caveat that must accompany the corollary.** The floor is *not* uniform over weights:  $\lambda \rightarrow 0$  drives it to 0 (numerically  $1.43 \rightarrow 0.19 \rightarrow 0.02$  for  $\lambda = 2, 0.2, 0.02$  at  $\mu = 5, \delta = 1$ ). Claiming “no weighting can succeed” requires bounding the weights away from zero. Ordered repair removes the *simultaneous-conflict term* only; it does not make total trajectory cost zero, and still requires reachability and feasible transitions.

## 6.2 Synthesis-layer results

**Proposition 1** (Abstract search soundness). *If the search expands  $o$  at  $s$  only when  $\text{pre}_o(s) = 1$  and returns  $\pi$  only when  $\text{sat}_G$  holds at the terminal abstract state, then every operator is applied in a state satisfying its declared precondition and the terminal state satisfies every requirement of  $G$ .*

*Scope:* this concerns the **abstract model only**. Effects are declared, not derived from dynamics; physical validity is established separately by rollout verification.

**Proposition 2** (Finite termination). *For finite  $S$ , finite  $O$ , all  $c_o > 0$ , and plan length bounded by  $L$ , uniform-cost search terminates. Each operator is used at most once and must change the state, giving a much smaller practical bound.*

**Proposition 3** (Conditional completeness). *If a goal-reaching plan exists in the bounded space  $\Pi_L = \{\pi : |\pi| \leq L, \pi \text{ applicable from } s_0\}$ , uniform-cost search returns one; with cost-optimal tie-breaking, one of minimum cost.*

*Scope:* completeness is relative to  $\Pi_L$  and to  $O$ . It is **not** open-set completeness.

**Proposition 4** (Coverage-complexity). *Under a stated model in which  $K$  predicates may hold in any combination, all  $2^K - 1$  combinations are reachable, and a precompiled architecture stores a complete branch per covered combination: full coverage requires  $\Theta(2^K)$  branches, whereas a compositional library of  $\Theta(K)$  operators covers the same combinations provided a goal-reaching plan exists in  $\Pi_L$ , at the cost of runtime search.*

*Scope:* this compares two *idealized* strategies. It does **not** claim every offline architecture is necessarily exponential — a real system may share subgraphs, parameterize branches, or cover only likely combinations.

**Proposition 5** (Conditional restoration). *If repair execution reaches a concrete state  $x$  with  $\text{Pre}(s_i^{\text{resumed}})(x) = 1$ , the resumed stage begins from a state satisfying its declared entry conditions.*

*Explicitly not claimed:* that the controller reaches such a state under model mismatch.

### 6.3 Three levels of validity

| Level     | Claim                                                  | Measured status                                                                                    |
|-----------|--------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <b>L1</b> | abstract plan soundness                                | holds by construction                                                                              |
| <b>L2</b> | rollout feasibility under the planning model $\hat{f}$ | precision 1.000 at zero mismatch                                                                   |
| <b>L3</b> | physical execution success under $f \neq \hat{f}$      | <b>degrades:</b> false-accept<br>0.000 $\rightarrow$ 0.028 $\rightarrow$ 0.063 $\rightarrow$ 0.228 |

No result in this project asserts **L1**  $\Rightarrow$  **L3** or **L2**  $\Rightarrow$  **L3**. L2 is the bridge; the mismatch study quantifies how leaky it becomes.

## 7 Code architecture

| Package          | Responsibility / key entities                                                                                                                                                      | Implements             |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| program/         | StageSpec, StageState, TaskProgram (state machine, interrupt_active_stage, splice_repair_before_successor, atomic_advance), Continuation, TaskGraph, contracts                     | Eq. (9); $\kappa_t$    |
| events/          | EventType, deterministic predicates, EventDetector, NoiseModel (delay / FP / FN)                                                                                                   | event predicates       |
| repair/          | AbstractState, RepairGoal, SYMBOLIC_OPS, RepairPlanner (search), SynthesisGenerator vs. CandidateGenerator, legality / feasibility filters, RolloutVerifier, Scorer, RepairManager | Props. A–D; Eq. (8)    |
| execution/       | ExecutionContext (carries the <i>current task goal</i> ), action.py (single clipping utility shared with the verifier), controllers, guard monitor, Executor, fallback, provenance | shared harness         |
| policies/        | fixed / path-only / safe-stop / template / offline (branch & parameterized) / online / verifier variants / ablations                                                               | the compared methods   |
| synthetic/       | SceneConfig, Synthetic2DEnv, Rollout (SD, CFR, TS), conflict_bound                                                                                                                 | Thm. 1                 |
| benchmark/       | scenes (3 families, stable seeding), runner (paired grid), statistics, mismatch, probe                                                                                             | §9                     |
| adapter.py       | frozen RepairEnvironmentAdapter + SyntheticAdapter                                                                                                                                 | simulator independence |
| rekep_adapter.py | ReKepOmniGibsonAdapter (deferred GPU imports; dry-run fake)                                                                                                                        | §13                    |



## 7.1 End-to-end call graph

```

Executor.run(policy)
loop t = 0 .. horizon
  policy.act(ctx) [policies/online_repair.py]
  _maybe_advance(ctx)
    stage_exit(...) [execution/guard_monitor.py]
    _validate_restore(ctx) -> mark_restore_validated
    TaskProgram.advance() ATOMIC; may raise RestorationBlocked
  detector.detect(ctx) [events/detector.py]
  _plan_and_splice(event, ctx) only if repair required
  Continuation(...) capture kappa
  env.predicates(d_react) SAME channel for every policy
  state_from_predicates / goal_from_predicates
  RepairManager.plan(...)
    SynthesisGenerator.generate -> RepairPlanner.plan SEARCH
    LegalityFilter.check acyclic + restore + attachment
    FeasibilityFilter.check cheap reachability screen
    RolloutVerifier.verify SEQUENTIAL; hard reject
    Scorer.select Eq. (select)
  TaskProgram.interrupt_active_stage(kappa) -> SUSPENDED
  TaskProgram.splice_repair_before_successor()
  control(behavior, ctx, stage, kappa, params, ctx.task_goal)
  env.step(action) -> clip_action(...) [execution/action.py] SHARED

```

## 8 Bugs and research-validity issues already resolved

Each of these changed a conclusion, not merely the code.

| Issue                                                        | Defect                                                                                                    | Why it mattered scientifically                                                                      |
|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Hand-coded disconnected <code>TaskProgram</code> solver from | The first synthetic result compared two hand-written trajectory solvers; the program IR was never invoked | The headline result did not test the claimed mechanism at all                                       |
| Non-atomic restore transition                                | <code>advance()</code> marked the node <code>COMPLETED</code> before checking the restore gate            | A <i>blocked</i> transition still mutated state, violating the state machine’s central invariant    |
| Duplicate runtime repair IDs                                 | A second repair reused <code>suspend#0</code> , corrupting order and making the graph cyclic              | Multi-interruption episodes — the interesting case — were silently invalid                          |
| Template lookup vs. true synthesis                           | <code>TEMPLATES[event]</code> stored complete sequences; the “synthesis” claim was instantiation          | The paper’s central novelty claim was not implemented                                               |
| Asymmetric offline baseline                                  | Offline saw only one event class and had no runtime verification                                          | The compound-event “win” was a baseline artifact; with a fair baseline it disappears (exact parity) |
| Stale relocation goal                                        | Controllers and metrics used the immutable configured goal                                                | Relocation episodes scored <i>success</i> while ending 0.30 from the moved goal                     |

| Issue                                       | Defect                                                                                                         | Why it mattered scientifically                                                                           |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Stabilize while ungrasped                   | Compound plans stabilized a released object                                                                    | Physically meaningless; corrected preconditions changed the synthesized order                            |
| Inconsistent action clipping                | Environment clipped per axis ( $\sqrt{2} v_{\max}$ diagonally), verifier clipped the norm                      | The verifier predicted trajectories the world could not execute                                          |
| Sequential vs. simultaneous compound events | “Compound” scenes fired the second event during repair, where it was deferred, so predicates never co-occurred | There was never a union goal to compose; the composition ablation had nothing to ablate                  |
| Observation-noise wiring                    | <code>MismatchedEnv.step()</code> returned the <i>inner</i> env’s clean context                                | Every observation-side mismatch dimension was silently bypassed; the first attribution ranking was wrong |
| Unused delay parameters                     | <code>detector_delay</code> / <code>guard_delay</code> declared, never used                                    | Two mismatch sources reported exactly zero damage for a wiring reason                                    |
| Python salted <code>hash()</code>           | Scene sampling keyed on <code>hash(str)</code> , salted per process                                            | The benchmark was <b>not reproducible across processes</b>                                               |
| SD vs. TS conflation                        | Safe delivery was reported as task success                                                                     | Overstated end-to-end capability by $\sim 0.29$ absolute                                                 |

## 9 CPU experimental protocol

**Task families.** `upright_transport` (carry upright to a goal), `constrained_place` (tighter tolerance, slower limits), `align_insert` (tightest orientation requirement).

**Event conditions.** `obstacle`, `slip`, `relocation`, and the compound `obstacle+slip`, `obstacle+relocation`, `slip+relocation`. In compound conditions the predicates are forced to co-occur at the same detection instant, so the repair goal is genuinely a union.

**Severities.** `low`, `medium`, `high`, scaling obstacle radius/duration, drop displacement, and relocation magnitude.

**Paired randomized scenes.** `sample_scene(spec)` is a pure function of (family, condition, severity, seed) using a process-stable BLAKE2B seed, so pairing is exact *by construction* and reproducible across processes. Randomized: initial pose, goal, obstacle placement/radius/timing, slip time and drop displacement, relocation direction and magnitude.

**Scale.** Main benchmark  $3 \times 6 \times 3 \times 50 = 2,700$  episodes per method  $\times 6$  methods = **16,200** episodes. Ablations **9,720**. Aggregate mismatch **17,280**. Per-dimension attribution **16,740**.

**The three success metrics — never conflated.**

SD : object delivered to the *current* goal, no collision; (11)

CFR : no collision and no safe-fallback; (12)

TS : all terminal constraints (goal, orientation, danger-weighted spill, no collision). (13)

**Statistics.** Wilson intervals for proportions; paired bootstrap ( $10^4$  resamples) for differences; *exact* McNemar for paired binary outcomes; Holm correction across each comparison family. Effect sizes are reported, not only  $p$ -values.

## 10 CPU results

### 10.1 Strong-baseline comparison

Table 3: Main benchmark, 16,200 paired episodes. Wilson 95% CI.

| Method               | Safe delivery               | Collision-free recovery     | Task success                |
|----------------------|-----------------------------|-----------------------------|-----------------------------|
| path-only            | 0.436 [0.418, 0.455]        | 0.776 [0.759, 0.791]        | 0.182 [0.168, 0.197]        |
| safe-stop            | 0.623 [0.604, 0.641]        | 0.962 [0.954, 0.968]        | 0.266 [0.249, 0.283]        |
| template repair      | 0.429 [0.411, 0.448]        | 0.427 [0.408, 0.445]        | 0.262 [0.246, 0.279]        |
| offline atomic       | 0.593 [0.574, 0.611]        | 0.593 [0.574, 0.611]        | 0.423 [0.404, 0.442]        |
| <b>offline exact</b> | <b>0.973 [0.966, 0.978]</b> | <b>0.973 [0.966, 0.978]</b> | <b>0.683 [0.665, 0.700]</b> |
| <b>online repair</b> | <b>0.973 [0.966, 0.978]</b> | <b>0.973 [0.966, 0.978]</b> | <b>0.683 [0.665, 0.700]</b> |

Table 4: Paired difference vs. online repair (bootstrap 95% CI; exact McNemar, Holm-adjusted). **Offline exact is exactly equal on all three metrics.**

| Method               | $\Delta$ safe delivery         | discordant (+/-) | $p_{\text{Holm}}$ |
|----------------------|--------------------------------|------------------|-------------------|
| path-only            | +0.537 [+0.518, +0.556]        | 1450 / 1         | $< 10^{-4}$       |
| safe-stop            | +0.350 [+0.332, +0.369]        | 948 / 2          | $< 10^{-4}$       |
| template repair      | +0.544 [+0.525, +0.563]        | 1468 / 0         | $< 10^{-4}$       |
| offline atomic       | +0.380 [+0.362, +0.399]        | 1026 / 0         | $< 10^{-4}$       |
| <b>offline exact</b> | <b>+0.000 [+0.000, +0.000]</b> | <b>0 / 0</b>     | <b>1.0000</b>     |

Table 5: Safe delivery by condition. The offline-atomic gap is concentrated entirely in compound events.

| Condition           | path-only | safe-stop | template | off. atomic | off. exact | online |
|---------------------|-----------|-----------|----------|-------------|------------|--------|
| obstacle            | 0.431     | 0.984     | 0.984    | 0.984       | 0.984      | 0.984  |
| slip                | 0.042     | 0.042     | 1.000    | 1.000       | 1.000      | 1.000  |
| relocation          | 1.000     | 1.000     | 0.018    | 1.000       | 1.000      | 1.000  |
| obstacle+slip       | 0.293     | 0.673     | 0.573    | 0.573       | 0.864      | 0.864  |
| obstacle+relocation | 0.802     | 0.987     | 0.000    | 0.000       | 0.989      | 0.989  |
| slip+relocation     | 0.049     | 0.049     | 0.000    | 0.000       | 1.000      | 1.000  |

### 10.2 Coverage-storage trade-off

### 10.3 Ablations

**Measured causal mechanisms.** *No synthesis* costs  $-0.535$  safe delivery (compound-event SD collapses  $0.952 \rightarrow 0.209$ ). *No rollout verification* raises the collision rate  $0.005 \rightarrow 0.032$  ( $\approx 6\times$ ). *No continuation contract* costs  $-0.070$  SD and raises handoff error  $0.075 \rightarrow 0.126$ .

Table 6: Coverage vs. stored structure over all 7 predicate combinations ( $K = 3$ ). Online stores a constant operator library and compiles nothing.

| Method            | combos | branches | graph nodes | compile expansions | stored             | SD coverage |
|-------------------|--------|----------|-------------|--------------------|--------------------|-------------|
| offline budget(1) | 1      | 1        | 5           | 6                  | 1 branch           | 1/7         |
| offline atomic(3) | 3      | 6        | 32          | 31                 | 3 branches         | 5/7         |
| offline exact(7)  | 7      | 17       | 109         | 157                | 7 branches         | 7/7         |
| <b>online</b>     | —      | —        | <b>0</b>    | <b>0</b>           | <b>9 operators</b> | <b>7/7</b>  |

Table 7: Runtime cost (means over 2,700 episodes/method). Offline latency *includes* one-off precompilation performed at reset; in deployment that is amortized. **expand** is the online-only search cost.

| Method          | latency (ms) | repairs | repair len | cand. gen | cand. rej | search expansions |
|-----------------|--------------|---------|------------|-----------|-----------|-------------------|
| path-only       | 0.40         | 0.00    | 0.00       | 0.00      | 0.00      | 0.00              |
| safe-stop       | 0.68         | 0.00    | 0.00       | 0.00      | 0.00      | 0.00              |
| template repair | 1.37         | 0.52    | 2.35       | 1.40      | 0.62      | 0.00              |
| offline atomic  | 1.75         | 0.81    | 4.33       | 1.69      | 0.03      | 0.00              |
| offline exact   | 4.17         | 1.40    | 7.85       | 3.40      | 0.04      | 0.00              |
| online repair   | 2.95         | 1.40    | 7.85       | 3.73      | 0.04      | <b>15.16</b>      |

Table 8: Ablations, 9,720 paired episodes. Wilson 95% CI.

| Variant                           | Safe delivery               | Task success | collide      | fail-safe | handoff err  |
|-----------------------------------|-----------------------------|--------------|--------------|-----------|--------------|
| <b>full</b>                       | 0.972 [0.961, 0.980]        | 0.679        | 0.005        | 0.028     | 0.075        |
| No synthesis                      | <b>0.427</b> [0.398, 0.457] | 0.254        | 0.005        | 0.574     | 0.454        |
| No rollout verification           | 0.964 [0.951, 0.973]        | 0.679        | <b>0.032</b> | 0.004     | 0.060        |
| No continuation contract          | <b>0.908</b> [0.890, 0.924] | 0.582        | 0.005        | 0.092     | <b>0.126</b> |
| <i>No event composition</i>       | 0.972 [0.961, 0.980]        | 0.679        | 0.005        | 0.028     | 0.075        |
| <i>No restore gate</i>            | 0.972 [0.961, 0.980]        | 0.679        | 0.005        | 0.028     | 0.075        |
| <i>No state-dependent scoring</i> | 0.972 [0.961, 0.980]        | 0.679        | 0.005        | 0.028     | 0.075        |

**The three nulls, and why they are null.** These were investigated by *internal-activation tracing* (does the ablation move the intended variable?) and *targeted counterfactuals*. All three are **internally active but outcome-neutral**:

- **Event composition.** The reduced goal genuinely differs in 163/216 compound episodes — yet plans, selections, outcomes, candidate rejections and search expansions are identical in 0/216. *Cause*: operator preconditions already encode the composition (*Realign* requires  $\text{safe\_clearance} \wedge \neg \text{obstacle\_present} \wedge \text{object\_grasped} \wedge \text{object\_stable}$ ), so a single-predicate goal cannot be reached without resolving the others.  $\Rightarrow$  *representation property*.
- **Restore gate. Zero rejections in 706 validations**, including under *severe* model mismatch: *Realign* establishes the contract before the transition is attempted.  $\Rightarrow$  *safety invariant that always holds*, not a source of gain.
- **State-dependent scoring.** Selection is genuinely contested ( $> 1$  surviving candidate in 79.2% of decisions; selection changes in 25/432 episodes) yet yields **zero** outcome differences — every rollout-verified candidate is outcome-equivalent.  $\Rightarrow$  *plan-quality preference*.

Per our own protocol, the benchmark was **not** redesigned to force these significant. Event composition and state-dependent scoring can be removed with no measured loss; the restore gate is retained as a cheap invariant.

## 10.4 Model mismatch

Table 9: Aggregate mismatch, 17,280 episodes. False-accept rate = accepted by the verifier, failed physically.

| Verifier                    | none         | mild         | moderate     | severe       |
|-----------------------------|--------------|--------------|--------------|--------------|
| none (accept all)           | 0.034        | 0.067        | 0.094        | 0.253        |
| <b>nominal</b>              | <b>0.000</b> | <b>0.028</b> | <b>0.063</b> | <b>0.228</b> |
| conservative                | 0.000        | 0.027        | 0.054        | 0.214        |
| receding horizon            | 0.000        | 0.028        | 0.064        | 0.226        |
| false-reject (nominal)      | 0.080        | 0.080        | 0.080        | 0.080        |
| false-reject (conservative) | <b>0.667</b> | <b>0.623</b> | <b>0.507</b> | <b>0.304</b> |

Nominal verification is dependable through *mild*, marginal at *moderate*, and unreliable at *severe* (precision 0.772). Neither mitigation works: conservative margins barely reduce false accepts while causing massive false rejects and *worse* net safe delivery (0.887 vs. 0.918 at moderate); receding-horizon reverification is within noise despite  $\sim 2.9$  replans per episode. **Both are excluded from the claims.**

**Conclusion. Observation-side mismatch dominates; actuation-gain error is comparatively mild.** The top three sources are all cases where the robot’s *belief* about where or when something is wrong. Actuation error is absorbed by closed-loop feedback. One nuance: detector delay is nearly harmless for safe delivery (0.961 vs. 0.970) but materially damages *task success* (0.522 vs. 0.681), because a late repair leaves less horizon to restore orientation — SD and TS rank the sources differently.

The honest scope condition for the verification claim:

Sequential rollout verification is reliable to the extent that the observed state is accurate and current; it is insensitive to actuation-model error and sensitive to observation latency and position noise.

Table 10: Per-dimension attribution at *severe* (16,740 episodes); each source enabled *alone*.  $\Delta$ SD relative to the all-nominal baseline (SD = 0.970).

| Mismatch source                    | $\Delta$ safe delivery | false accept | collide |
|------------------------------------|------------------------|--------------|---------|
| <b>guard (observation) delay</b>   | <b>+0.367</b>          | 0.393        | 0.054   |
| <b>object-position noise</b>       | <b>+0.280</b>          | 0.285        | 0.009   |
| <b>target-position error</b>       | <b>+0.178</b>          | 0.183        | 0.004   |
| stochastic regrasp failure         | +0.135                 | 0.139        | 0.013   |
| release drift                      | +0.044                 | 0.046        | 0.009   |
| action noise                       | +0.041                 | 0.042        | 0.004   |
| detector delay                     | +0.009                 | 0.002        | 0.015   |
| obstacle-position prediction error | +0.006                 | 0.006        | 0.009   |
| translation gain bias              | +0.004                 | 0.004        | 0.004   |
| rotation gain bias                 | +0.000                 | 0.000        | 0.004   |

## 11 Frozen paper claims

| Claim                                       | Status    | Evidence                                                                                                                | Scope                                                      |
|---------------------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| <b>1. Runtime operator synthesis</b>        | primary   | −0.535 SD when ablated; compound SD 0.209 vs. 0.952                                                                     | novel <i>composition</i> of known predicates, not open-set |
| <b>2. Sequential rollout verification</b>   | primary   | collisions 0.032 vs. 0.005; FA 0.000 at zero mismatch                                                                   | reliable only insofar as observation is accurate/current   |
| <b>3. Continuation-carrying restoration</b> | primary   | −0.070 SD; handoff error 0.126 vs. 0.075                                                                                | restoration <i>conditional on arrival</i>                  |
| <b>4. Coverage-storage trade-off</b>        | primary   | exact parity (0/0, $p = 1.0$ ); +0.380 under partial coverage; 9 operators vs. 17 branches / 109 nodes / 157 expansions | under a stated combination model                           |
| Event composition                           | secondary | goal differs 163/216, outcome 0/216                                                                                     | representation property                                    |
| Restore gate                                | secondary | 0 rejections / 706                                                                                                      | safety invariant                                           |
| State-dependent scoring                     | secondary | contested 79.2%, outcome 0                                                                                              | plan-quality preference                                    |
| Provenance logging                          | secondary | no behavioral path                                                                                                      | instrumentation                                            |

**Explicitly not claimed.** Arbitrary open-set recovery; universal superiority over precompiled recovery; guaranteed physical safety under model mismatch; a learned world model; real-robot validation; conservative or receding-horizon verification.

## 12 Current limitations

- **Synthetic kinematic environment.** 2-D point robot, first-order integration, no inertia or contact physics; one scalar orientation.

- **Simplified attachment/regrasp.** Regrasp completes on proximity; no gripper state or grasp quality.
- **Imperfect correspondence to real ReKep stages.** Our `StageSpec` assumes sub-goal+path constraints and a guard; ReKep’s actual structure may diverge (risk R7).
- **Rollout model assumptions.** The verifier uses the nominal model;  $L2 \neq L3$ .
- **Observation-error sensitivity.** The dominant damage source (up to  $\Delta SD = 0.367$ ).
- **Severe-mismatch false accepts** reach 0.228.
- **OmniGibson nondeterminism** may weaken the paired-seed protocol.
- **GPU integration not executed.**
- **Scene-specific keypoint mapping** — adapter indices are placeholders.
- **Simulator runtime cost** — verification calls ReKep solvers  $\sim 18$  times per repair.
- **Task-success ceiling** 0.683 for both online and offline exact: residual failures come from the nominal controller and strict terminal criteria, not from recovery.

### 13 GPU phase plan

Every command below is **REMOTE GPU SERVER ONLY**. Nothing has been executed.

**Milestone 0 — installation.** Verify driver ( $\geq 525.60$ ) and GPUs; create Python 3.10 env; install `torch==2.2.2+cu121` *first* from the CUDA index; install OmniGibson (brings Isaac Sim 2023.1.1) and its assets; install ReKep; run `check_gpu_environment.py`.

**Milestone 1 — nominal parity.** Run one official ReKep task *unchanged* with cached queries; record the stage sequence and constraints; wrap/mirror it in `TaskProgram`; verify wrapped execution is behaviorally *identical*. **Do not modify ReKep solvers.** Any divergence is a wrapping bug and contaminates every later comparison.

**Milestone 2 — first repaired episode.** Inject one deterministic temporary obstacle; use simulator ground-truth predicates initially; capture the continuation; synthesize; verify; splice; restore; resume; save trace and video. The CPU dry-run already produces the expected program shape:

```
pour -> ...:r1::suspend#0 -> retreat#1 -> stabilize#2
      -> waituntilclear#3 -> realign#4 -> resume#5 -> pour#resume::r1
```

**Milestone 3 — task suite.** Upright transport with obstruction; alignment/insertion with target relocation; placement with slip; compound interruption as a later extension. No fluid simulation initially.

**Milestone 4 — noisy perception.** Keypoint noise, predicate hysteresis, observation latency, target uncertainty — prioritized by the attribution table, which says these dominate.

**Milestone 5 — multi-GPU study.** Only after single-GPU reproducibility. The launcher refuses to start without the Milestone-1 gate file.

## 14 ICRA experiment matrix

Minimum viable.

|             |                                                                          |
|-------------|--------------------------------------------------------------------------|
| Tasks       | 3 (upright transport, insertion+relocation, placement+slip)              |
| Event types | 3 single (obstacle, slip, relocation)                                    |
| Compound    | 1 (obstacle+slip)                                                        |
| Severity    | 2 (moderate, high)                                                       |
| Methods     | 4: ReKep fixed recovery, path-only, offline exact parameterized, REKEP-R |
| Ablations   | 3: NoSynthesis, NoRolloutVerification, NoContinuationContract            |
| Seeds       | 20 paired per cell                                                       |
| Metrics     | SD, CFR, TS, collision, handoff error, latency, FA rate                  |
| Statistics  | Wilson, paired bootstrap, exact McNemar, Holm                            |

Approximately  $3 \times 4 \times 2 \times 4 \times 20 = 1,920$  simulator episodes plus ablations.

**Stretch.** All 6 compound conditions; 3 severities; offline budget sweep for the coverage-storage curve in-simulator; noisy-perception study (Milestone 4); latency breakdown (detect / synthesize / verify / solve); failure gallery with videos; 50 seeds per cell.

## 15 Handoff checklist

For the collaborator, on the GPU server, in order:

1. Read `README_FIRST.md`, then `KNOWN_RISKS.md` (**before** installing — R1 alone can cost days).
2. `python scripts/check_gpu_environment.py --dry-run` — repo and CPU stack sanity, no GPU imports.
3. `python -m pytest` inside `code/` — confirm 71 tests pass on your machine.
4. Follow `GPU_RUNBOOK.md` steps 1–6 (inspect, env, deps, assets, checker, OmniGibson smoke test).
5. Edit `configs_gpu/` placeholders (`<REKEP_ROOT>`, `<OMNIGIBSON_ASSET_PATH>`, `<OUTPUT_ROOT>`) and the adapter keypoint indices — these are certain to be wrong by default.
6. Runbook steps 7–9: nominal → wrapped → repaired ReKep.
7. Runbook steps 10–11: inspect traces/videos; approve or reject Milestone 1. On approval, `touch runs/milestone1/GATE_PASSED`.
8. Only then consider multi-GPU sweeps.

Next go/no-go criterion.

One nominal ReKep episode and one online-repaired ReKep episode execute *reproducibly* on a single RTX 3090 using the *same* underlying ReKep solvers.